

Introduction

Purpose:

Developing travel plans can be overwhelming and difficult as the information is often fragmented across multiple platforms. This approach can be especially challenging if you are an inexperienced traveler. With Journey, travelers get personal insights into potential activities for their travel destination in the form of “experiences”. Travelers can then organize “experiences” of interest into a personalized trip itinerary. Journey provides travelers with the benefit of a centralized location for information and recommendations, ultimately leading them to explore more activities and enjoy a richer travel experience.

Team Members:

- Allison Langlois - Frontend
- Kevin Lin - Backend
- Michael Valderrama - Backend
- Sean Miller - Frontend

Project Overview

High Level Tech Stack

Frontend: React

Backend: Python (FastAPI)

Database: MongoDB

Media Storage: Cloudinary

Testing: pytest

Authentication: JWT (HTTP-only cookies)

Deployment: Google Cloud Run, Docker

Tooling: Git, Swagger UI

Live Application:

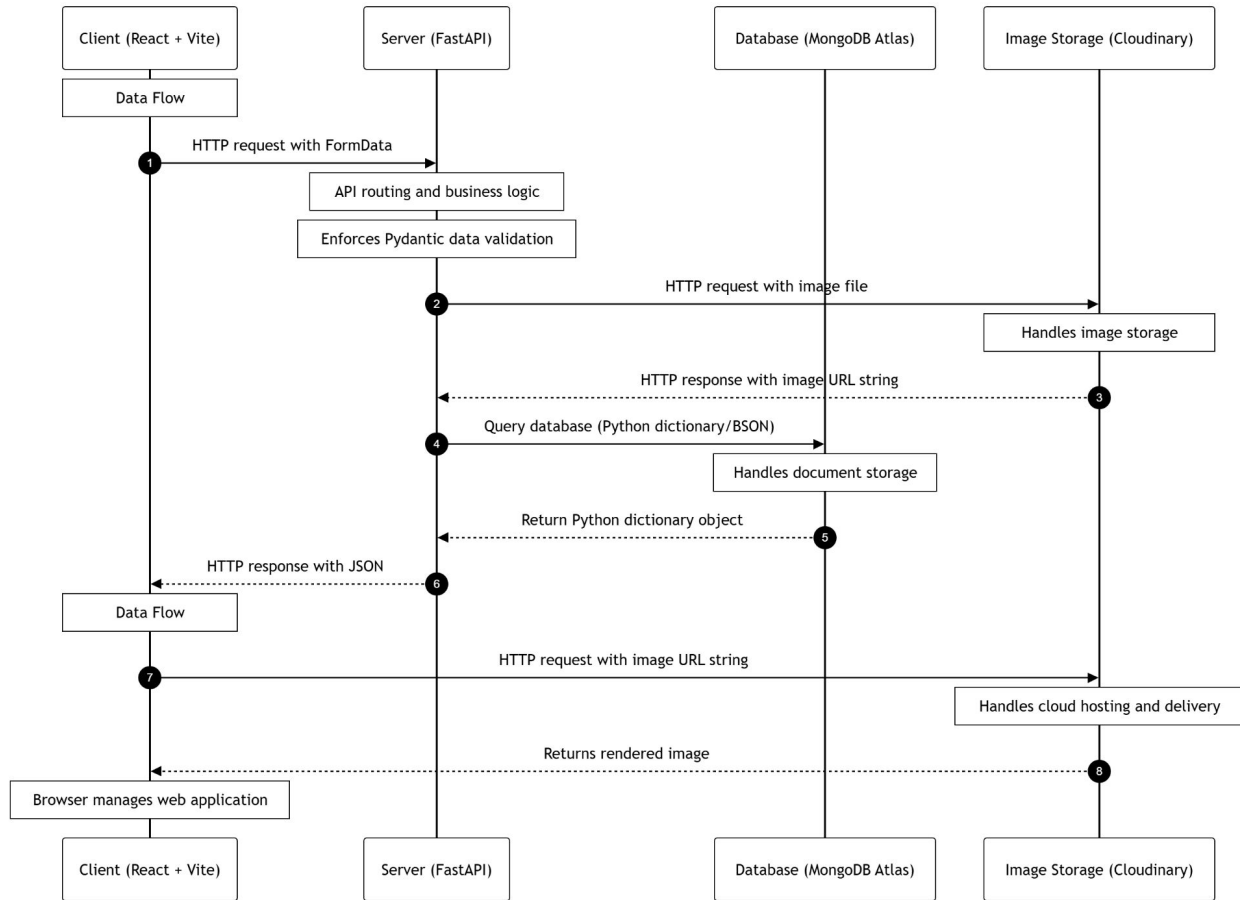
<https://travelplanner-129337949325.us-central1.run.app/home>

API Documentation (Swagger UI):

<https://travelplanner-129337949325.us-central1.run.app/docs>

GitHub Repository:

<https://github.com/travel-project-team/Crowd-Sourced-Travel-Planner>



Highlights

- **Dedicated Search**
 - Search by location or keyword across all user created experiences.
- **User Collaborations**
 - Share trip itineraries with other Journey users.
 - View experience ratings from other travelers.
- **Experience Creation**
 - Log new experiences with photo uploads, automatic geolocation capture, and star ratings.
- **Trip Creation**
 - Organize experiences into a trip itinerary.

Development - Allison

- Project Responsibilities

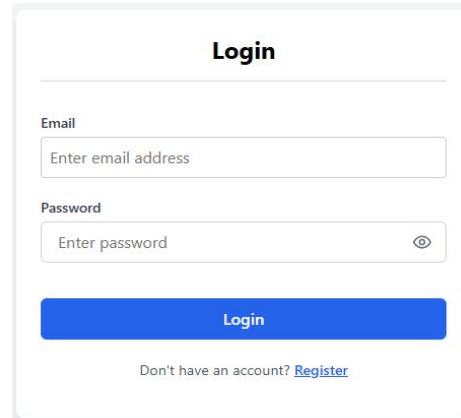
- Implemented the Login and Registration pages
- Developed the common page layout, which included a sidebar navigation and header
- Developed the user profile pages - the user edit profile page and change password page
- Added a button for deleting a user's account
- Developed the page routing layout - used JWT token to distinguish public vs private pages
- Usability testing - implemented changes based on usability test results

Development - Allison

- Technologies Used
 - React
 - Framework used to set up initial frontend environment and user interface
 - JavaScript
 - Wrote the code for the getting the user's initials in JavaScript.

Development - Allison

- Sprint 1: Login & Registration Pages, initial Homepage and Page Routing
 - Designed and implement the Login and Registration pages
 - Implemented initial page routing - restricted the Homepage so it is only seen after a user is successfully authenticated



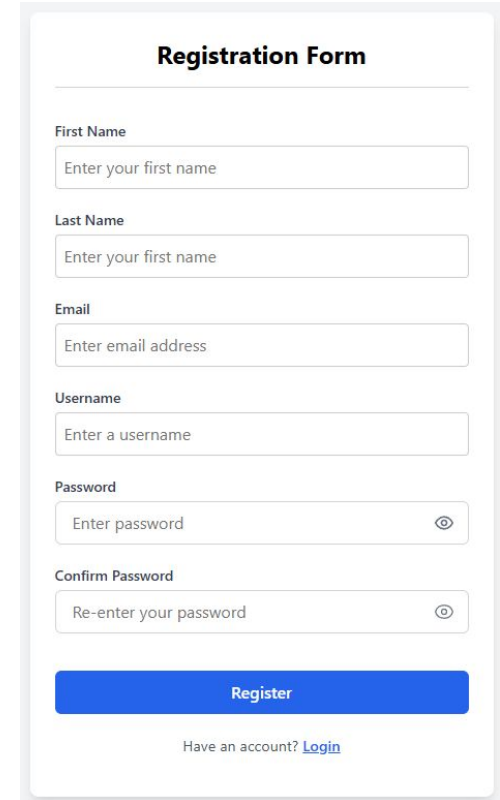
Login

Email

Password

[Login](#)

Don't have an account? [Register](#)



Registration Form

First Name

Last Name

Email

Username

Password

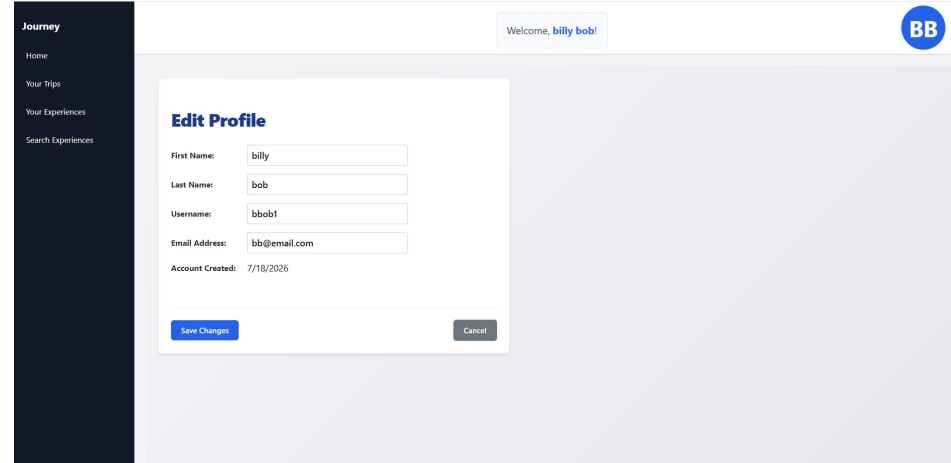
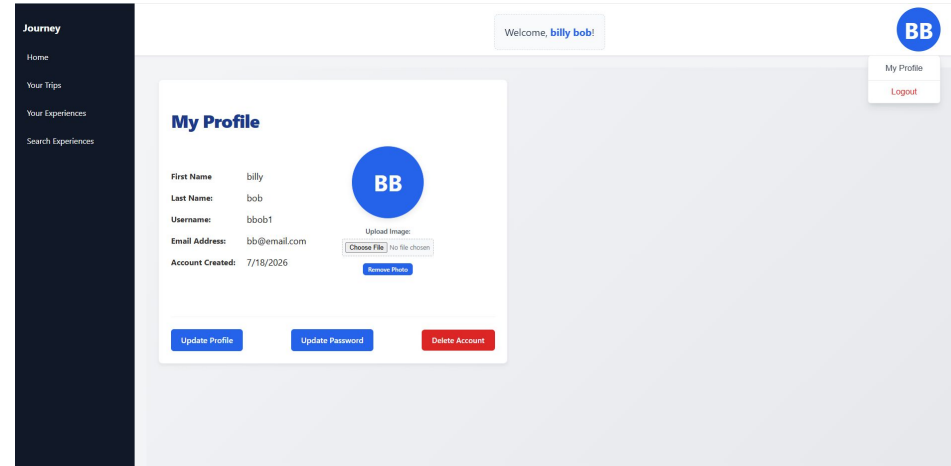
Confirm Password

[Register](#)

Have an account? [Login](#)

Development - Allison

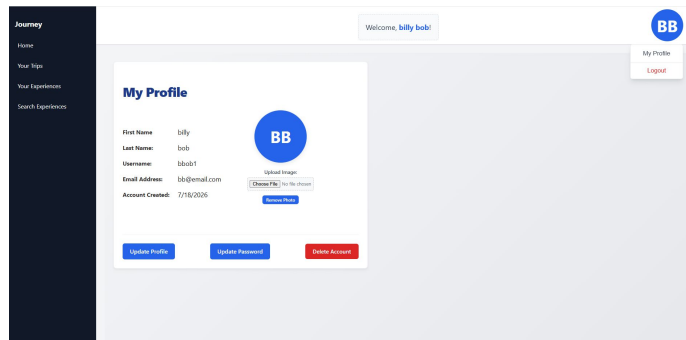
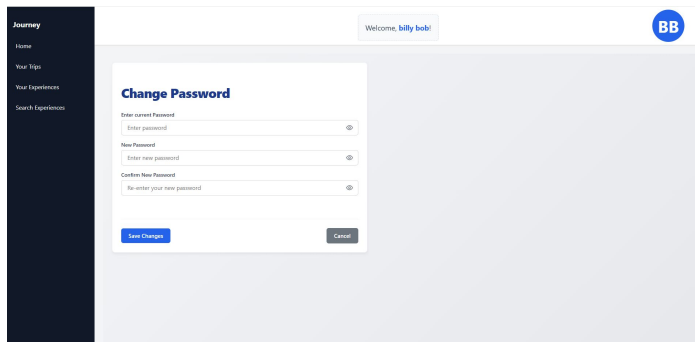
- Sprint 2: Common Page Layout, Logout Button Functionality, User Profile Page, and Update Profile Page
 - Developed a common page layout that included a sidebar for navigation and header
 - Implemented a logout button feature
 - Designed and implemented a user profile page
 - Designed and implemented an update profile page



Development - Allison

- Sprint 3: User Profile Image Functionality, User Change Password Page, Reorganization of Page Routing and Frontend Files
 - Implemented functionality that allowed the user to upload and set a profile image
 - Implemented a change password page that is accessible from the user's profile
 - Reorganized the routing of pages so the user initially lands on the homepage instead of the login page
 - Reorganized the folder structure of the frontend files

```
30 function App() {
31   return (
32     <>
33       <Router>
34         <ScrollToTop />
35         <Routes>
36           { /* Public Routes */ }
37           <Route element={<LoginPage />} />
38           <Route path="/login" element={<LoginPage />} />
39           <Route path="/register" element={<Registration />} />
40           { /* Protected Routes */ }
41           <Route element={<ProtectedRoutes />} />
42           <Route path="/profile" element={<Profile />} />
43           <Route path="/profile/edit" element={<EditProfile />} />
44           <Route path="/profile/change-password" element={<ChangePassword />} />
45           <Route path="/trips" element={<Trips />} />
46           <Route path="/trips" element={<AddExperience />} />
47           <Route path="/experiences" element={<Experiences />} />
48           <Route path="/single-experience/:id" element={<SingleExperience />} />
49           <Route path="/add-trip" element={<AddTrip />} />
50           <Route path="/edit-trip/:id" element={<EditTrip />} />
51           <Route path="/edit-experience/:id" element={<EditExperience />} />
52           <Route path="/search" element={<SearchPage />} />
53         </Route>
54       </>
55       { /* Catch-all fallback redirect */ }
56       <Route path="*" element={<Navigate to="/home" replace />} />
57     </Routes>
58   </Router>
59 )
60 }
```



```

  > frontend
  > node_modules
  > src
  > assets
  > components
    > ExperienceCard.jsx
    > Header.jsx
    > Sidebar.jsx
    > TripCard.jsx
  > helpers
  > JS
    > getInitials.js
    > ProtectedRoutes.jsx
    > ScrollToTop.jsx
  > layouts
    > CommonPageLayout.jsx
  > pages
    > AddExperience.jsx
    > AddTrip.jsx
    > ChangePassword.jsx
    > EditExperience.jsx
    > EditProfile.jsx
    > EditTrip.jsx
    > Experiences.jsx
    > HomePage.jsx
    > Login.jsx
    > Profile.jsx
    > Registration.jsx
    > SearchPage.jsx
    > SingleExperience.jsx
    > Trips.jsx
  > services
  > JS
    > api.js
  > styles
    > App.jsx
    > global.css
  > main.jsx
```

Development - Kevin

- Project Responsibilities

- Set up backend FastAPI and frontend React environment
- Built core user backend APIs
- Implemented Cloudinary SDK connection for image handling
- Automated CI/CD workflow for development and production

Development - Kevin

- Technologies Used

- FastAPI
 - Framework used to build backend REST APIs
- React
 - Library used to set up initial frontend environment
- Cloudinary SDK
 - Integrated for handling user photo uploads
- Vite
 - Build tool and development server for frontend
- Google Cloud Run
 - Hosts and deploys the containerized application

Development - Kevin

- Sprint 1: Project Set-Up & User Registration
 - Defined technology stack and configured environments
 - Handled user authentication
 - Implemented Login and Registration API calls

```
# Account Registration (usersApi.create(data))
@router.post("", status_code=status.HTTP_201_CREATED)
def create_user(user_info: UsersRegister):
    """
    Input: JSON with user information.
    Output: JSON with success/error message.
    """

    # Check for existing email and username
    if config.db.users.find_one({"email": user_info.email}):
        raise HTTPException(status_code=400, detail="Email already registered")
    if config.db.users.find_one({"username": user_info.username}):
        raise HTTPException(status_code=400, detail="Username already taken")

    hashed = hash_password(user_info.password)
    new_user = {
        "first_name": user_info.first_name,
        "last_name": user_info.last_name,
        "username": user_info.username,
        "email": user_info.email,
        "password_hash": hashed,
        "avatar_url": None,
        "created_at": datetime.now(timezone.utc)
```

```
# Generate JWT Access Token
def create_access_token(data: dict) -> str:
    """
    Input: User information to put into JWT payload.
    Output: Signed JWT access token with an expiration.
    """

    user_information = data.copy()

    # Add token expiration
    expiration_time = datetime.now(timezone.utc) + timedelta(minutes=EXPIRATION_MINUTES)
    user_information.update({"exp": expiration_time})

    # Access token claims
    # sub: email
    # user_id: _id
    # expiration_time: default 60 minutes
    jwt_token = jwt.encode(user_information, SECRET_KEY, algorithm=ALGORITHM)

    return jwt_token

# Verify Access Token
def verify_user(request: Request):
    """
```

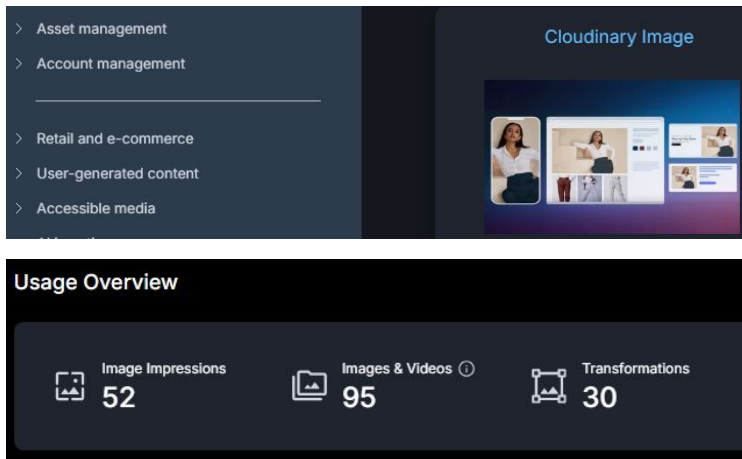
Directory Structure

travel_planner

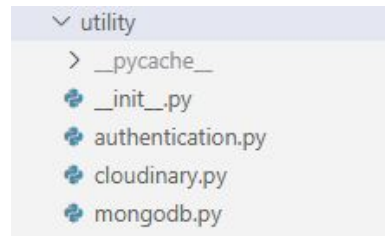
- backend (FastAPI)
 - .venv --Local virtual environment
 - .env --Secret_key, mongoDB, and cloudinary credentials
 - .env.example --env template
 - requirements.txt --Dependencies
- src
 - routes --CRUD
 - init.py
 - users.py
 - experiences.py
 - trips.py
 - schemas --Validate and format database documents
 - init.py
 - users.py
 - experiences.py
 - trips.py
 - utility --Helper files for routes

Development - Kevin

- Sprint 2: Image Storage & User API Routes
 - Initialized image upload and storage via Cloudinary
 - Implemented backend User API routes
 - Added utility files for api, mongodb, and cloudinary



```
/* =====  
| Users  
===== */  
export const usersApi = {  
  create: (data) => api.post("/users", data),  
  
  login: (data) => api.post("/users/login", data),  
  
  logout: () => api.post("/users/logout"),  
  
  getProfile: () => api.get("/users"),  
  
  update: (data) => api.put("/users", data),  
  
  updatePassword: (data) => api.put("/users/password", data),  
  
  remove: () => api.delete("/users"),  
}
```



Development - Kevin

- Sprint 3: Security Upgrades & Deployment
 - Upgraded access token to HTTP cookies
 - Added API routes for expanded features
 - Automated deployment to Google Cloud Run

All checks have passed ×

1 successful check

 **Capstone-Project-Travel-Planner (capstone-project-503000)** Successful in 3m - [Details](#)

 [Summary](#)

Triggers + Create trigger → Connect repository 🗑️ Manage repositories					
A Cloud Build trigger automatically starts a build whenever you make any changes to your source code or some other incoming event.					
≡ Filter Enter property name or value					
Name ↑	Region	Description	Repository	Event	Build configuration
Capstone-Project-Travel-Planner	global	—	 travel-project-team/Crowd-Sourced-Travel-Planner 🔗	Push to branch	cloudbuild.yaml

```
// Include HTTP authentication cookies with every request
try {
  const response = await fetch(url, {
    ...options,
    signal: controller.signal,
    credentials: "include",
    headers: {
      ...(options.body instanceof FormData
        ? {}
        : { "Content-Type": "application/json" }),
      ...options.headers,
    },
  });
}
```

```
getBatchById: (data) => api.post("/users/id", data),
getBatchByEmail: (data) => api.post("/users/email", data),
uploadAvatar: (data) => api.post("/users/avatar", data),
removeAvatar: () => api.delete("/users/avatar"),
```

Development - Michael

- Project Responsibilities

- Designed the application's centralized HTTP request layer with timeout handling and standardized error management.
- Developed the Experiences and Trips backend REST APIs, including search, permissions, and supporting features.
- Built the automated backend testing framework using pytest and an in-memory MongoDB test environment.
- Redesigned the homepage.
- Developed the Experience Search page.

Development - Michael

- Technologies Used

- FastAPI
 - REST API framework with dependency injection and request validation
- MongoDB Atlas
 - Document database storing experiences, trips, and user links
- React Router
 - Client-side routing for protected and public pages
- pytest + mongomock
 - Test framework and in-memory database for the backend test suite
- Cloudinary
 - Image URLs accepted and stored by experiences endpoint

Development - Michael

- Sprint 1: Request Layer & Core APIs
 - Built a reusable frontend HTTP request layer with centralized timeout and error handling.
 - Designed and implemented CRUD REST endpoints for Experiences and Trips.
 - Added keyword, location, and query parameter support for searching experiences.
 - Implemented owner and collaborator permission logic for trip management.

```
// Convert params object to a query string and skipping empty values. Example: buildQuery({ location: "Seattle", limit: 10 }) --> "?location=Seattle&limit=10"
function buildQuery(params = {}) {
  const query = new URLSearchParams();
  Object.entries(params).forEach(([key, value]) => {
    if (value !== undefined && value !== null && value !== "") {
      query.append(key, value);
    }
  });
  const qs = query.toString();
  return qs ? `?${qs}` : "";
}
```

Development - Michael

● Sprint 2: Expanded Endpoints & Testing

- Integrated image uploads, ratings, and trip-to-experience linking.
- Added pagination and sorting support to improve the scalability of list endpoints.
- Established the backend testing framework using pytest, shared fixtures, and mongomock.
- Implemented cascade account deletion to maintain database consistency.

```
22 @pytest.fixture()
23 def test_db(monkeypatch):
24     """Swap config.db for an in-memory mongomock database."""
25     mock_client = mongomock.MongoClient()
26     monkeypatch.setattr(config, "db", mock_client.travel_planner_test)
27     return config.db
28
29 @pytest.fixture()
30 def client(test_db):
31     """A TestClient bound to the app, with the mock database already in place."""
32     return TestClient(app)
33
34 @pytest.fixture()
35 def make_user(test_db):
36     """Factory fixture for creating users; call the returned function per user needed."""
37
38     def _make_user(
39         username: str = "bill11111123",
40         email: str = "bill11111123@example.com",
41         password: str = "hunter21",
42         first_name: str = "Billie",
43         last_name: str = "Billerson",
44     ):
45         """Insert a user with a hashed password and return their details plus auth cookie."""
46         result = test_db.users.insert_one(
47             {
48                 "first_name": first_name,
49                 "last_name": last_name,
50                 "username": username,
51                 "email": email,
52                 "password_hash": hash_password(password),
53                 "avatar_url": None,
54                 "created_at": datetime.now(tz=timezone.utc),
55             }
56         )
57         user_id = str(result.inserted_id)
58         token = create_access_token({"sub": email, "user_id": user_id})
59         return {
60             "id": user_id,
61             "username": username,
62             "email": email,
63             "password": password,
64             "first_name": first_name,
65             "last_name": last_name,
66             "token": token,
67             "cookies": {"access_token": token},
68         }
69     return _make_user
70
71 @pytest.fixture()
72 def default_user(make_user):
73     """A single pre-registered user for tests that just need someone logged in."""
74     return make_user()
```

```
PS C:\Users\miche\OneDrive\Work\Travel-Planner\backend> pytest
platform win32 -- Python 3.10.8, pytest 8.4.1, pluggy 1.6.0
rootdir: C:\Users\miche\OneDrive\Work\Travel-Planner\backend
configfile: pytest.ini
testpaths: tests
plugins: anyio 4.10.1
collected 8 items

tests/test_experiences.py .....
tests/test_trips.py .....
tests/test_users.py .....

===== warnings summary =====

www/lib/site-packages/fastapi/testclient.py:1
  C:\Users\miche\OneDrive\Work\Travel-Planner\backend\www\lib\site-packages\fastapi/testclient.py:1: StarletteDeprecationWarning: Using 'http' with 'starlette.testclient' is deprecated, install 'httpx' instead.
  from starlette.testclient import TestClient as TestClient # deprecated

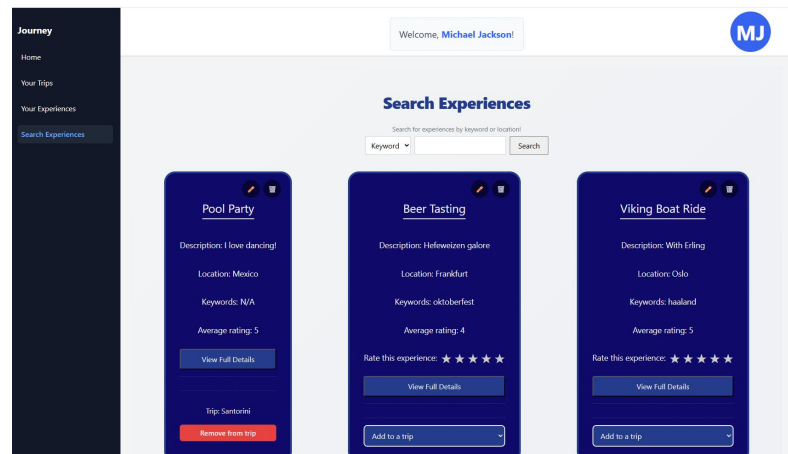
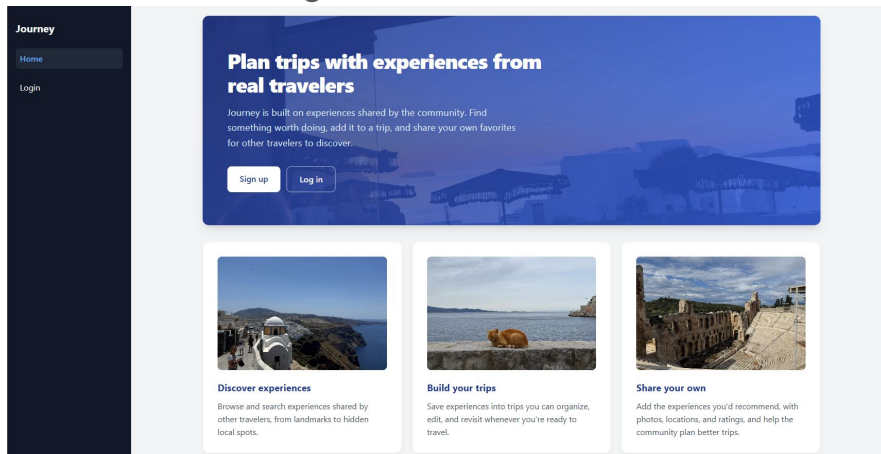
tests/test_experiences.py: 41 warnings
tests/test_trips.py: 48 warnings
tests/test_users.py: 37 warnings
  C:\Users\miche\OneDrive\Work\Travel-Planner\backend\www\lib\site-packages\starlette/testclient.py:464: DeprecationWarning: Setting per-request cookies {...} is being deprecated, because the expected behavior on cookie persistence is ambiguous. Set cookies directly on the client instance instead.
    return super().request()

tests/test_experiences.py: 41 warnings
tests/test_trips.py: 48 warnings
tests/test_users.py: 37 warnings
  C:\Users\miche\OneDrive\Work\Travel-Planner\backend\www\lib\site-packages\starlette\testclient.py:464: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    now = timegm(datetime.utcnow().utctime())

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 8 passed, 295 warnings in 22.16s =====
```

Development - Michael

- Sprint 3: Homepage redesign, Search Experience page, bug fixes
 - Implemented the Search Experience page - user can search for experiences by keyword or location.
 - Redesigned the homepage with a modern hero section and feature cards.
 - Fixed the "Your Experiences" page so it only displays content created by user.
 - Expanded project documentation by improving the README and adding docstrings throughout the backend test suite.



Development - Sean

- Project Responsibilities

- Drew up the schema and configured MongoDB for data persistence.
- Coded the initial Pydantic schemas on the backend.
- Designed and implemented all frontend features relating to CRUD operations for trips and experiences.
- Assisted with frontend bug fixes

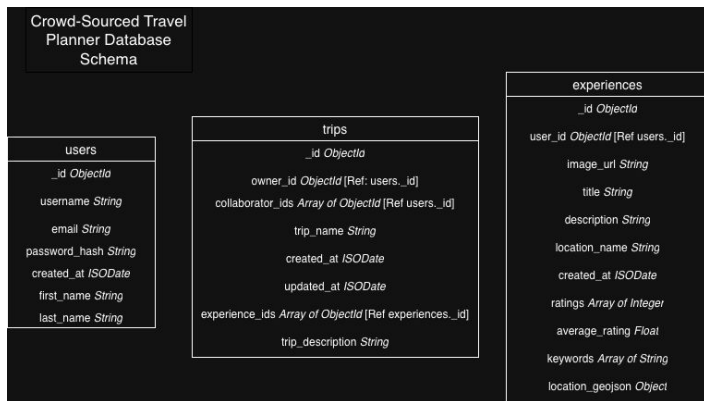
Development - Sean

- Technologies Used

- JavaScript
 - Wrote the code for the frontend exclusively in JavaScript.
- React
 - Used for the frontend framework.
- Python
 - Wrote the Pydantic schemas in Python and referenced the backend routes regularly for frontend development.
- MongoDB Atlas
 - Used the web-based UI to setup initial data in the database and referenced regularly to ensure proper data persistence.
- Cloudinary
 - Used to upload and store images that were displayed on the frontend.

Development - Sean

- Sprint 1: Database Setup and Initial Trips/Experiences Pages
 - Setup the MongoDB cluster and shared it with the group.
 - Crafted the schema for the database.
 - Wrote the initial Pydantic schemas.
 - Started the early stage version of the Trips and Experiences frontend components.



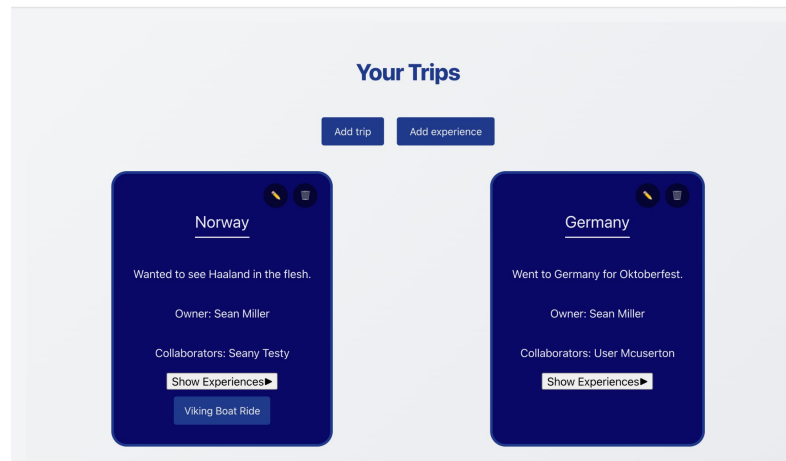
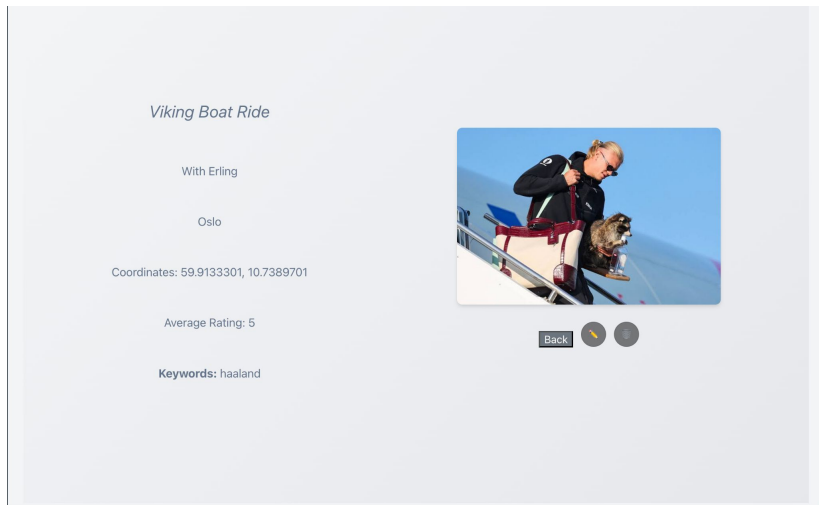
TravelPlannerCluster > travel_planner

[View monitoring](#) [Visualize your data](#) [+ Create collection](#) [Refresh](#)

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
experiences	-	36.86 kB	7.73 kB	22	351.00 B	1	36.86 kB
trips	-	36.86 kB	4.21 kB	15	281.00 B	1	36.86 kB
users	-	36.86 kB	4.60 kB	19	241.00 B	1	36.86 kB

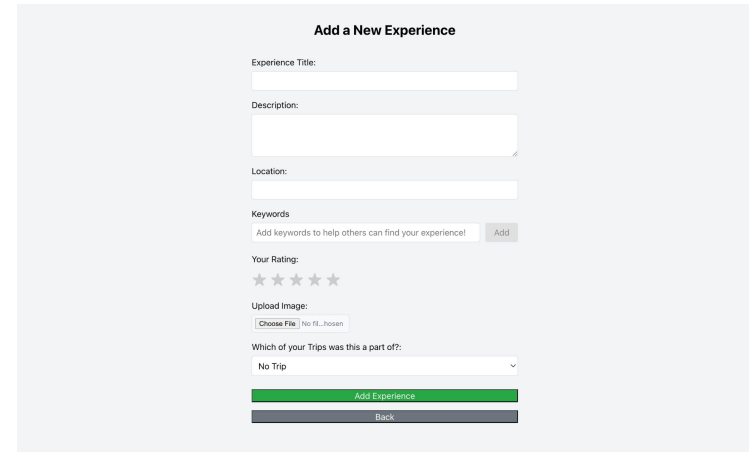
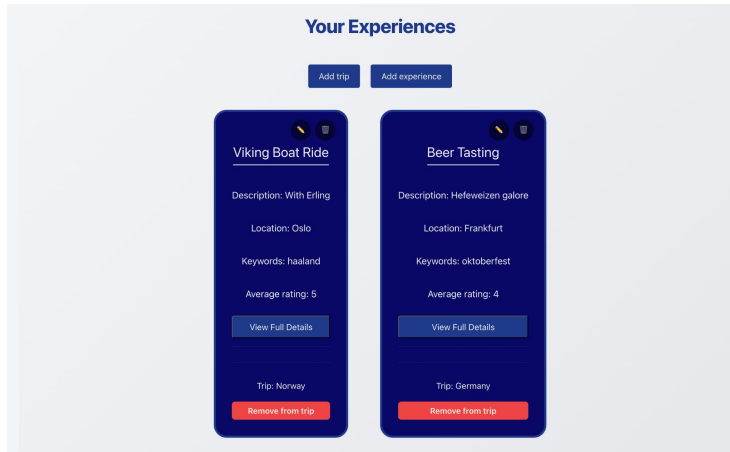
Development - Sean

- Sprint 2: Completing the Initial UI and Starting the Backend Connection
 - Completed the initial version of the UI for all essential components regarding Trips and Experiences.
 - Began the initial connections with actual backend data as opposed to stand-in data drew from arrays on the frontend.



Development - Sean

- Sprint 3 - Accomplishing Full Functionality for Trips/Experiences Frontend
 - Completed the UI and backend connection for all components relating to Trips and Experiences.
 - Ensured all core objectives related to these components were complete.
 - User acceptance testing and bug fixes for the frontend.



Conclusion

- Where do things go from here? / Future Development
 - We plan on adding a new feature where a user can enter their destination address and view experiences on a map within a 25 mile radius around that location.
 - We plan on expanding the search functionality so users can search by calendar date
- Thank you
 - We would like to thank Professor Hedao for supporting our project and clarifying questions on Ed Discussion
 - We would like to thank Professor Pfeil for the project idea